

TensorFlow:

<https://www.tensorflow.org/>

INTRODUCTION



Étapes préparatoires

Desktop	Raspberry Pi (seulement Tensorflow lite)
	<pre>\$ pip install numpy # math \$ pip install pandas # analyse de données \$ pip install scikit-learn # apprentissages</pre>
N/A	<u>Référence</u> : https://projects.raspberrypi.org/en/projects/getting-started-with-picamera/0 <pre>\$ sudo raspi-config # --> 3. Interface Options --> I1 camera --> Oui</pre> Si vous n'avez pas de caméra USB
pip install tensorflow	<u>Référence</u>: https://pimylifeup.com/raspberry-pi-tensorflow-lite/ <pre>\$ sudo apt update \$ sudo apt upgrade -y \$ echo "deb [signed-by=/usr/share/keyrings/coral-edgetpu-archive-keyring.gpg] https://packages.cloud.google.com/apt coral-edgetpu-stable main" sudo tee /etc/apt/sources.list.d/coral-edgetpu.list \$ curl https://packages.cloud.google.com/apt/doc/apt-key.gpg sudo tee /usr/share/keyrings/coral- edgetpu-archive-keyring.gpg >/dev/null \$ sudo apt update \$ sudo apt install python3-tflite-runtime libatlas-base-dev</pre> ** Possiblement obsolète, voir plus bas
pip install lobe	<u>Référence</u>: https://learn.adafruit.com/machine-learning-101-lobe-braincraft/get-predictions-on-the-pi <pre>\$ pip install lobe \$ wget https://raw.githubusercontent.com/lobe/lobe-python/master/scripts/lobe-rpi-install.sh \$ sudo bash lobe-rpi-install.sh</pre>

**** Référence:** <https://learn.adafruit.com/running-tensorflow-lite-on-the-raspberry-pi-4/tensorflow-2-setup>

```
RELEASE=https://github.com/PINTO0309/Tensorflow-bin/releases/download/v2.10.0/tensorflow-2.10.0-cp310-none-linux_aarch64.whl
CPVER=$(python --version | grep -Eo '3\.[0-9]{1,2}' | tr -d '.')
pip install $(echo "$RELEASE" | sed -e "s/cp[0-9]\{3\}/CP$CPVER/g")
```



TensorFlow en Action



Reconnaissance
faciale

*La voie de droite est libre.
Changer de voie*



Véhicule auto-
pilotant

TensorFlow

Une interface pour exprimer des algorithmes d'AA et une implémentation pour les exécuter.

Un cadre informatique pour créer de nouveaux algorithmes d'AA pour résoudre des problématiques modernes



TensorFlow

Pourquoi le nom TensorFlow?

Toutes les données que vous souhaitez traiter avec TensorFlow doivent être stockées dans des tableaux multidimensionnels. Ces tableaux multidimensionnels sont également appelés tenseurs. Pour exécuter des opérations sur l'ensemble de données, vous construisez une séquence de calcul similaire à un organigramme qui détermine comment les données circulent (flow) d'une opération à l'autre.



Classification catégorielle avec TensorFlow

pizza_categories.txt

Une donnée catégorielle possède un nombre fini de valeurs possibles:

Température: [chaud,froid]

Seuil-rentabilité: [0, 1]

Couleurs: [rouge, vert, bleu, orange]

Réervations	Température	Touristes	Seuil-rentabilité
13	chaud	9	1
2	froid	6	0
14	chaud	3	1
23	chaud	9	1
13	chaud	8	0
13	chaud	9	1
1	chaud	7	0
18	froid	2	1
7	chaud	3	0
10	chaud	3	0
26	froid	7	1
3	chaud	1	0
3	froid	4	0
21	chaud	2	1
22	froid	4	1
2	chaud	7	0
27	froid	3	1
6	chaud	8	0
10	froid	6	0
18	chaud	9	1
15	chaud	10	1
9	chaud	7	0
26	froid	2	1
8	chaud	1	0
15	froid	7	0
10	froid	9	0
21	froid	4	1
5	froid	3	0
6	froid	3	0
13	chaud	4	1



Classification catégorielle avec Tensorflow

pizza_categories.py

```
1 import pandas as pd # analyse de données
2 from sklearn.model_selection import train_test_split # AA
3 from tensorflow.keras.models import Sequential, load_model # AA
4 from tensorflow.keras.layers import Dense
5 from sklearn.metrics import accuracy_score
6 import tensorflow as tf
7
8 fd = pd.read_csv('pizza_categories.txt')
9 # on omet la cible et on transforme les traits non-numériques
10 X = pd.get_dummies(fd.drop(['Seuil-rentabilité'], axis = 1)) # les traits
11 y = fd['Seuil-rentabilité'] # la cible catégorique
12 # on divise nos données
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = .2)
14
15 # Ici, on choisit et paramétrise notre modèle: voir https://www.tensorflow.org/
16 #=====
17 model = Sequential()
18 model.add(Dense(units=32, activation = 'relu', input_dim = len(X_train.columns)))
19 model.add(Dense(units=64, activation='relu'))
20 model.add(Dense(units=1, activation = 'sigmoid'))
21 model.compile(optimizer='sgd', loss = 'binary_crossentropy', metrics='accuracy')
22
23 # On le produit, epochs est le nombre d'itérations
24 # model.fit(X_train, y_train, epochs = 200)
25 model.fit(tf.convert_to_tensor(X_train , dtype=tf.int32),
26           tf.convert_to_tensor(y_train , dtype=tf.int32), epochs = 200)
27
28 # on demande les prédictions pour X_test
29 y_pred = model.predict(tf.convert_to_tensor(X_test , dtype=tf.int32))
30 # On mappe [0,0.5] à 0 et 1 autrement
31 y_pred = [0 if val < 0.5 else 1 for val in y_pred]
32
33 print("La vérité:")
34 print(y_test) # la vérité
35 print("La prédiction:")
36 print(y_pred) # la prédiction
37
38 print("Exactitude: ", round(accuracy_score(y_test, y_pred),2))
39
40 model.save('modele_pizza')
41
42 model = load_model('modele_pizza')
```



Classification catégorielle: modèle et résultat

```
C:\USERS\MICHE\DOWNLOADS\MODELE_PIZZA
├── fingerprint.pb
├── keras_metadata.pb
├── saved_model.pb
├── assets
├── variables
│   ├── variables.data-000000-of-00001
│   └── variables.index
```

La vérité:

29	1
27	0
23	0
25	0
10	1
8	0

Name: Seuil-rentabilité

La prédiction:

[1, 0, 1, 0, 1, 1]

Exactitude: 0.67



Lobe: <https://www.lobe.ai/>

Permet de créer des modèles TensorFlow pour la reconnaissance d'images

[Tutoriel](#)



Modèle et code Python pour la classification de chiens et chats: animal.zip

Structure de fichiers créée par Lobe

```
labels.txt
saved_model.pb
signature.json

-example
  classifier.py
  README.md
  requirements.txt

-test
  a1.jpeg
  a2.jpeg
  a3.jpeg

-variables
  variables.data-00000-of-00001
  variables.index
```



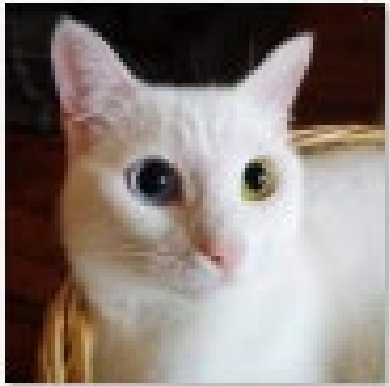
Contenu du fichier classifier.py

```
classifier.py X
1 from lobe import ImageModel
2
3 model = ImageModel.load("../")
4
5 for i in range(3):
6     result = model.predict_from_file('test/a'+str(i+1)+'.jpeg')
7     for animal, confidence in result.labels:
8         print("Animal " + str(i+1) + f" {animal}: {confidence*100}%")
```

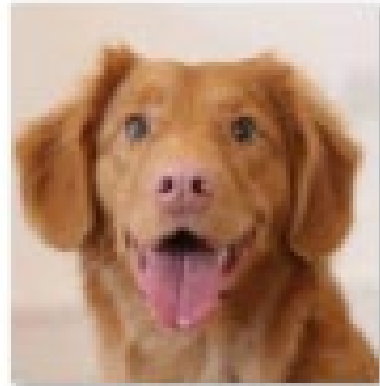
Ce modèle permet de classifier des images de chiens et de chats (a1, a2 et a3).



Résultat de la classification



✓ a1.jpeg



✓ a2.jpeg



✓ a3.jpeg

```
Animal 1 chats: 57.06534385681152%  
Animal 1 chiens: 42.934662103652954%  
Animal 2 chiens: 82.42712020874023%  
Animal 2 chats: 17.572884261608124%  
Animal 3 chiens: 89.75003361701965%  
Animal 3 chats: 10.249968618154526%
```

Modèle LOBE et code Python sur Raspi avec une caméra: happy_sad.zip

```
1 import cv2
2 from lobe import ImageModel
3 noImage = 1
4 model = ImageModel.load('./modele')
5 capture = cv2.VideoCapture(0)
6 while True:
7     _, img = capture.read()
8     cv2.imshow('Frame',img)
9     if cv2.waitKey(1) & 0xFF == ord('c'): # attendre la touche 'c'
10         nomFichier = "./img" + str(noImage) + ".jpg"
11         print("Capture #" + str(noImage) + " terminée.")
12         resultat = model.predict_from_file(nomFichier)
13         # L'étiquette de la prédiction, on l'inscrit sur l'image
14         etiquette = resultat.prediction
15         # Le niveau de confiance pour le meilleur choix
16         confiance = resultat.labels[0][1]
17         # Résultats
18         print(f"Prédiction: {etiquette} | Confiance: {confiance * 100: .2f}")
19         cv2.putText(img, f"{etiquette} | {confiance * 100: .2f}", (0,100), cv2.FONT_HERSHEY_COMPLEX,1,(255,255,255),2)
20         cv2.imwrite(nomFichier, img)
21         noImage += 1
22     if noImage > 3:
23         break
24 capture.release()
25 cv2.destroyAllWindows()
```

